

Article

A Logic-Based Framework for Detecting Inconsistencies of UML Models

Enas Naffar ^{1,*}, Amjad Hudaib ², Nadim Obeid ³ and Said Ghoul ⁴ 

¹ Department of Computer Science, King Abdulla II School of Information Technology, The University of Jordan, Amman 11942, Jordan

² Department of Computer Information Systems, King Abdulla II School of Information Technology, The University of Jordan, Amman 11942, Jordan; ahudaib@ju.edu.jo

³ Department of Artificial Intelligence, The University of Jordan, Amman 11942, Jordan; obein@ju.edu.jo

⁴ Department of Software Engineering, Philadelphia University, Amman 19392, Jordan; sghoul@philadelphia.edu.jo

* Correspondence: ayn9210104@ju.edu.jo

Abstract

Software modeling involves creating multiple software models that represent different viewpoints of the system under development. These models complement one another and should remain coherent throughout the development process, as they represent a single system. Ensuring consistency across different models is crucial for building high-quality software within time and budget constraints. Existing research on consistency management primarily focuses on structural consistency and covers only a subset of UML models. In this paper, we propose a logic-based framework for detecting inconsistencies in basic UML models. We develop a comprehensive metamodel of basic UML models that addresses both structural and semantic relations among metamodel elements. Furthermore, we define new consistency rules for use within the framework to detect various types of inconsistency. The proposed framework is validated using a formal logic-based modeling language and evaluated using two case studies. Compared with existing approaches, the proposed framework provides more comprehensive coverage of inconsistencies. Experimental results demonstrate the effectiveness of the proposed framework in detecting various types of inconsistencies.

Keywords: software modeling; UML; model consistency; cross-model consistency



Academic Editor: Aneta
Poniszewska-Maranda

Received: 8 May 2026

Revised: 15 June 2026

Accepted: 16 June 2026

Published: 23 June 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Software modeling is an essential activity in software development. It encompasses the construction of software models that capture various aspects of the system. A model is an abstract representation of the system under study that reflects a particular viewpoint. A good model should be abstract, capturing only the essential details of the system, to facilitate its understanding and manage its complexity. Various models are used in software development to support the analysis and design activities before moving into the implementation. Software models are used to facilitate effective communication among stakeholders, i.e., customers and developers. They are also used to capture the business process as they can depict business workflows, varied decisions, and relationships among system components [1]. Multiple software models are used during different phases of the software development lifecycle; each model represents a different viewpoint of the system. Separating the representation into varied viewpoints, each with a different abstraction

level, helps manage the system's complexity and enables a smooth transition towards an operational system that satisfies the customer's needs.

The UML, which stands for Unified Modeling Language, is the industry standard for creating software models. It is a graphical modeling language that is used to visualize, specify, and document models of a software system [2]. UML models are the core artifacts of the software development process; they facilitate analysis and design activities and serve as the basis for validation and verification, enabling software developers to detect and resolve potential defects early. UML organizes its diagrams into two categories: structural and behavioral, which capture the system's static and dynamic aspects, respectively [2]. Even though each diagram reflects a certain viewpoint, all diagrams are interrelated and complementary; when integrated, they should form a coherent system.

UML diagrams, such as use case, class, state machine, activity, and sequence diagrams, are modeling artifacts used to represent the system throughout the development process, as depicted in Figure 1. This paper uses the terms use case, class, sequence, activity, and state models to refer to the corresponding UML modeling artifacts associated with their respective diagram types. User requirements are captured in the use case model, which consists of use diagrams accompanied by their textual descriptions. In many cases, an activity diagram is created to represent the workflow. The class diagram captures the system's structural aspects, including classes and their relationships. The sequence diagram illustrates the objects' interactions over time for specific use case scenarios. For some classes, the state diagram is needed to capture the states and transitions of their objects. These varied diagrams are not isolated; they depend on each other and are interrelated; hence, they must remain consistent throughout the development process [3] to develop a coherent system that meets customer needs.

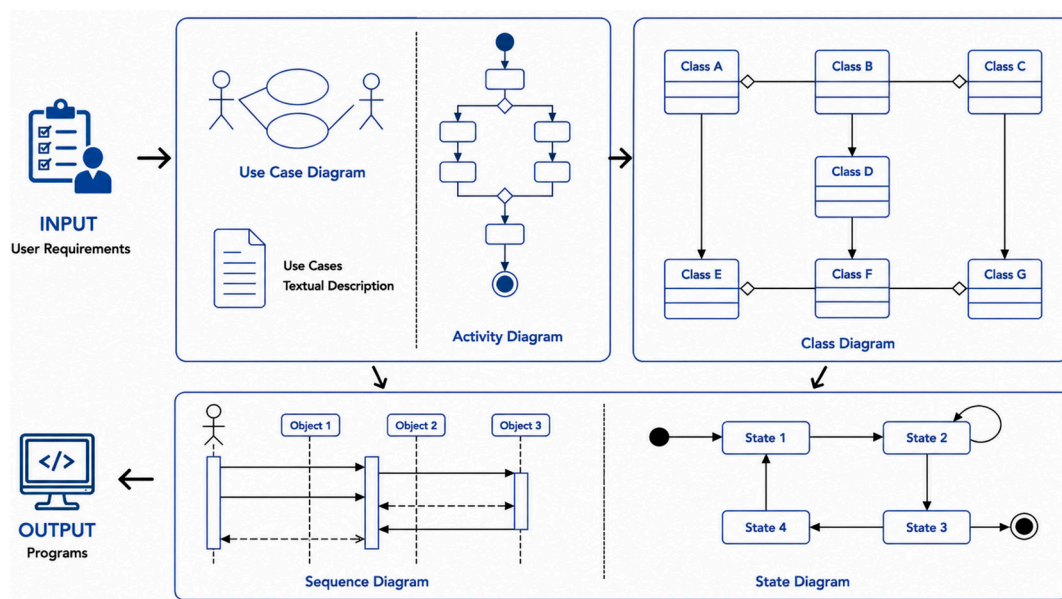


Figure 1. Different UML diagrams are used throughout the development. Standard UML notations are used for illustrative purposes.

Ensuring model consistency is critical for developing a reliable system. Consistency refers to the degree to which the model contains no conflicting requirements, assertions, constraints, functions, or component descriptions [4]. An inconsistency occurs when overlapping elements across different models do not match; as a result, the system is not represented harmoniously [5]. For example, a message represented in a sequence diagram may not be captured in a class diagram, or it may be depicted with a different number of parameters. Despite UML's support of comprehensive modeling, it does not inherently

support consistency checking across multiple models produced at different phases of software development; instead, cross-model consistency management is addressed by external frameworks and tools [6]. Therefore, without proper checking, an inconsistency may be apparent across multiple models that are supposed to reflect a single coherent system. Inconsistency among models can lead to conflicting interpretations of the system, resulting in defects, increased development time and costs, and potential system failures. As a result, a project may be terminated if it fails to meet the customer's needs [7].

As systems grow in size and complexity, manual checking of models' consistency becomes impractical, and the need for automated methods increases. Manual consistency management is a labor-intensive, time-consuming, and error-prone process [8]. A slight modification to one diagram may lead to many changes across the rest of the diagrams, making consistency management a challenging, error-prone task when performed manually. The considerable time and effort required to manually verify consistency across diagrams in every iteration of the analysis and design, or in the maintenance phase, slows down development, making manual consistency checking and preservation inefficient in practice [8]. Automated approaches, on the other hand, can detect inconsistencies early in the development process and significantly reduce the costs and risks associated with inconsistencies. Despite the availability of automated tools to check and maintain the consistency of UML models, such as IBM Rational Rose Enterprise, tool support remains limited [9], particularly in terms of accessibility and cost-effectiveness. Some advanced tools have extremely high licensing fees, making them unaffordable for educational institutions or projects with limited budgets and resources. There is a lack of comprehensive tool support for automated consistency checking and preservation across different models.

This paper examines current methods and techniques for maintaining UML models' consistency, identifies their limitations, and outlines opportunities for improvement. Current tools and research on consistency management of UML models have primarily focused on structural consistency across models, emphasizing syntactic correctness, such as element naming, attribute existence, operation existence, and multiplicity compatibility [6,7,10,11]. Less attention has been given to semantic consistency, which focuses on the intended meaning and the correspondence of elements to ensure coherent behavior. The literature also shows that many approaches cover only a restricted subset of UML models, such as class and sequence diagrams. In contrast, behavioral models, such as use cases and activity diagrams, have received less attention. While considerable efforts have been made in the field of consistency management of UML models, existing methods provide only partial support for comprehensive cross-model consistency verification. Most studies address specific consistency concerns or UML diagram types in isolation, limiting their applicability in complex software systems. Consequently, a unified framework is still needed to combine several UML models, explicitly represent cross-model relationships, and enable automated reasoning to detect inconsistencies. This is crucial for enhancing the reliability, maintainability, and correctness of software models across the software development lifecycle.

To address these limitations, we propose a logic-based framework for detecting inconsistencies across UML models (LBF-UML), defined at the metamodel level and applied at the model level. LBF-UML addresses both structural and semantic consistency by building a comprehensive metamodel with additional structural and semantic relations among metamodel elements. Furthermore, we introduce a new set of consistency rules, formally defined and specified, to enhance the identification of inconsistencies.

The proposed framework leverages automated reasoning and constraint-solving techniques to manage consistency detection across multiple UML models, improving the efficiency and flexibility of consistency checking and covering both structural and be-

havioral models. Overall, this study contributes to improving consistency management, enabling software developers to resolve conflicts in a way that saves time and effort and noticeably enhances software quality. Experimental results demonstrate the LBF-UML framework's ability to detect various types of inconsistencies in accordance with the defined consistency rules.

The rest of this paper is organized as follows: Section 2 outlines essential concepts in the literature and related work. Section 3 demonstrates our methodology for detecting inconsistencies of UML models, followed by the implementation details of LBF-UML in Section 4. Section 5 describes the experiment using two case studies to evaluate the framework. In Section 6, we demonstrate the findings and analyze the results. Eventually, Section 7 concludes the paper.

2. Literature Review

In this section, we outline the basic principles and terminology, which are fundamental to understanding our approach. These concepts provide the foundation of the proposed framework. We also present and discuss several studies to identify how the LBF-UML framework contributes to the body of existing knowledge compared to the techniques discussed below.

2.1. Background

In the following, we present core modeling concepts. We first introduce the notions of model and metamodel, then discuss metamodel layering. Furthermore, we present a brief background on Alloy.

2.1.1. Model vs. Metamodel

A metamodel defines the structure of a model, i.e., elements and their relationships, and the semantics of how elements of a model are instantiated. In contrast, a model is an instance of a metamodel and contains different model elements. A model can be a metamodel of another model [12]. Models often comprise one or more diagrams that graphically represent an aspect of the model or a subset of its elements.

OMG introduced a formal definition of MOF-based metamodel that specifies the syntax of the UML [12]. MOF, which stands for Meta-Object Facility, is a standard for specifying metamodels. It can be observed as the underlying metamodeling architecture for many modeling languages, such as the UML, as depicted in Figure 2. Hence, the UML metamodel can be considered an instance of the MOF metamodel, which defines, for example, how UML models are exchanged among tools using XML.

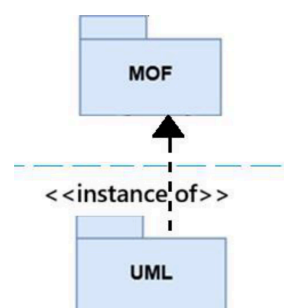


Figure 2. The relation between MOF and UML.

2.1.2. Metamodel Layering

In this section, we briefly explain the metamodeling architecture, a layered framework on which the UML metamodel is based. This architecture is composed of four layers:

- M3: This is the top-most layer, which represents the language for specifying the metamodel, like MOF. It specifies the core modeling constructs.
- M2: This layer represents instances of the top-most layer; hence, language specification or language metamodel exists in this layer, like UML.
- M1: This layer represents instances of modeling language; therefore, user specification or user models are found in this layer.
- M0: This is the lowest-level layer; it represents objects or instances of the user models.

Figure 3 illustrates the four-layer architecture of metamodeling, whereas Figure 4 demonstrates an example of the four-layer metamodeling architecture. In this study, the proposed metamodel extensions are defined at the M2 level, whereas the UML models analyzed for consistency are instances at the M1 level.

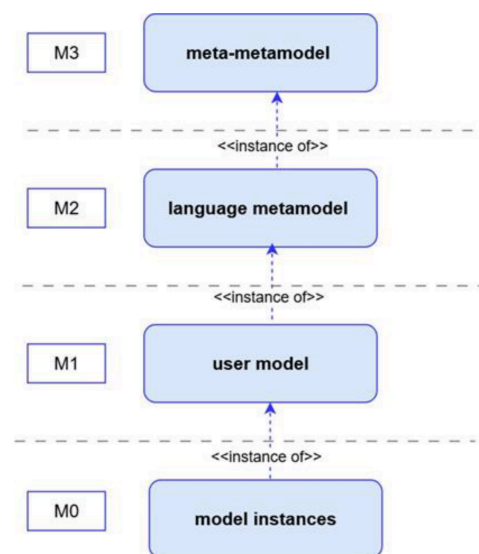


Figure 3. The four-level metamodeling architecture.

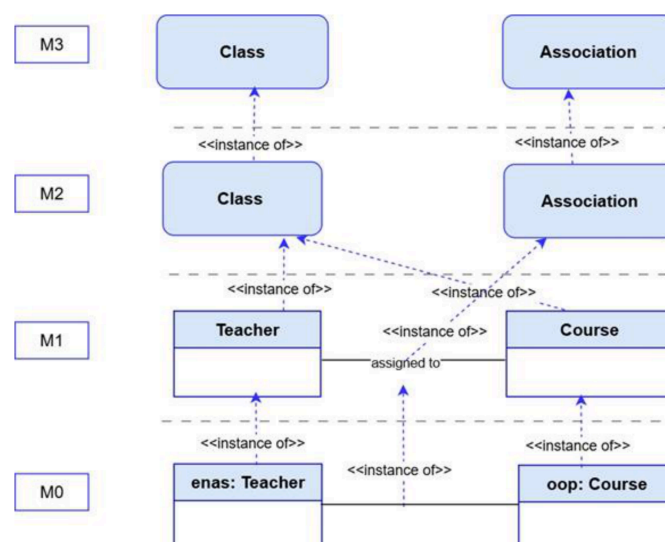


Figure 4. An example of the four-level metamodeling architecture.

2.1.3. Alloy

Alloy is a formal modeling language used for specifying and analyzing the structure of systems. Alloy language is based on first-order logic and the notion of relations, and is inspired by the Z specification language, Tarski's relational calculus, and the Linear Tempo-

ral Logic with Past [13]. Alloy supports automatic analysis via the Alloy Analyzer, which translates constraints into Boolean constraints for processing by a Boolean Satisfiability (SAT) solver [14]. Alloy is closely related to modeling languages, such as UML, and is influenced by object-oriented modeling notations [13,14]. In Alloy, the term model refers to the specification of a system, and the term instance denotes a satisfying valuation of that specification. This use of the term model in Alloy is consistent with UML, where a software model is an artifact denoted by a specification [15]. Alloy is used in this research to formally specify the proposed metamodels and consistency rules and to verify their logical consistency before their application to UML user models.

2.2. Related Work

Several studies in UML model consistency management have proposed techniques and frameworks to detect inconsistencies across UML models. Some of these studies have focused on identifying consistency rules within and across models. Whereas other studies have focused on finding a formal representation of UML models as a major step in managing their consistency. Research on automated consistency checking and preservation remains limited.

The study in [16] identified 119 existing consistency rules, documented in plain English. These rules have been obtained from varied sources and studies. The authors highlight the need for more research studies on consistency rules.

The paper in [17] proposed an approach to analyze the consistency among UML models by formally specifying them using a unified model structure. Three previously defined consistency rules have been used in their experimental tool. A similar approach is used in [11] to check consistency among UML models, but with a different set of consistency rules. A formal representation of UML diagrams has been used in both [11,17] to facilitate the analysis of UML models and hence detect inconsistencies in these models; however, these two approaches have not identified new consistency rules; instead, they have used some consistency rules that have been specified and documented in [16].

The work in [10] presented a case study involving 33 consistency rules and 34 UML models. This research was evaluated to assess how it would help detect inconsistencies across different phases of software development.

An integration of rule-based and Large Language Model (LLM) based methodologies is introduced in [18] to guarantee internal and cross-consistency among Systems Modeling Language (SysML v2) diagrams. It is limited to the consistency between use case diagrams and block diagrams.

The research in [19] used existing meta-models and integrated them into a single model to verify the consistency of requirements, Java, and class diagrams, focusing solely on the system's structural aspects. In [9], a prototype of a consistency-checking tool was proposed, based on user-defined consistency rules in the Abstract Rule Language (ARL). The study in [20] used existing consistency rules and grouped them into composite consistency rules to automate and simplify the construction of software architecture.

Alloy has been adopted by the Model-Driven Engineering (MDE) community for analyzing UML/Object Constraint Language (OCL) models, where most existing approaches transform UML/OCL models into Alloy to ensure consistency between system models and metamodels. The study in [21] enables users to specify metamodels, which are validated by integrating Alloy's first-order logic with the MOF (Meta-Object Facility). This approach facilitates reasoning about models' compliance with metamodels and hence allows structural inconsistencies to be detected at two levels: models and metamodels.

The authors of [22] proposed a formal approach to check a metamodel for inconsistencies by identifying conflicting constraints. This approach ensures that at least one

valid instance can be generated from the metamodel; if not, the metamodel is inconsistent. This approach is conceptually similar to Alloy’s reasoning; however, instead of relying on first-order logic, it combines OCL with the USE tool to validate the consistency at the metamodel level.

In [23], the authors emphasized the importance of validating system models, using the Peterson algorithm as a case study. Alloy was used as a formal validation technique in this paper, using the Alloy Analyzer as an automated tool to verify the model’s consistency and identify counterexamples. An integration between UML, a semi-formal language, and Alloy, a formal language, is proposed in [24], where a UML model is mapped to its equivalent Alloy model to facilitate automated analysis.

Table 1 provides a comparison of related work. The literature shows that many approaches address only a limited subset of UML diagrams, such as class and sequence diagrams. In contrast, behavioral aspects, represented by use cases and activity diagrams, have received less attention. Additionally, there is a significant variation among the existing approaches in their support for semantic consistency, metamodel integration, and automatic verification mechanisms. While several studies provide formal representations and verification capabilities, many rely on predefined rules and focus on specific model relationships. The proposed framework targets both structural and behavioral aspects, covering the following diagrams: use case, class, sequence, activity, and state. We define a comprehensive metamodel that integrates structural and semantic relations among basic UML models, where semantic relations capture the preservation of intended system behavior across different UML models of the same system. Additionally, we formally define new consistency rules, which are specified and validated in conjunction with the metamodel using Alloy.

Table 1. Summary of related work.

Research	Approach	Definition of New Rules	Models Covered	Structural Consistency	Semantic Consistency	Validation/ Experiment
[16]	Collecting and documenting 119 consistency rules	No	All	Yes	Yes	No
[17]	Building a formal representation of the models and using a few existing consistency rules	No	Class, Sequence, and State diagrams	Yes	Yes	Tool implementation
[11]	Building a formal representation of the models and using a few existing consistency rules	No	Class, Sequence, and State diagrams	Yes	Yes	Tool implementation
[10]	Formalizing some existing consistency rules, applying them to 34 UML diagrams	No	Class, Sequence, Activity, State, Use Case, Object, and Composite Structure diagrams	Yes	Yes	Using existing tools (Eclipse Papyrus and Eclipse OCL)

Table 1. Cont.

Research	Approach	Definition of New Rules	Models Covered	Structural Consistency	Semantic Consistency	Validation/ Experiment
[18]	Integrating a rule-based approach with LLM methodologies, handling consistency between use cases and block diagrams only	Yes	Use case and block diagrams	Yes	No	Tool implementation
[19]	Building a meta-model to check the consistency of the requirements, class diagram, and Java only.	No	Requirements model, class diagram, and code model (Java)	Yes	No	Tool implementation
[9]	Defining consistency checking rules by the user	No	Class, use case, sequence, and state diagrams	Yes	Partial	Tool implementation
[20]	Using composite consistency rules to construct systems' architecture automatically.	New composite rules using existing rules	Class, use case, sequence, and state diagrams	Yes	Yes	Tool implementation
[21]	Integrating Alloy with MOF to detect inconsistencies in any user-specified metamodel (model and metamodel levels)	No	MOF-based models	Yes	No	Tool implementation
[22]	Integrating OCL with the USE tool to detect inconsistencies in any user-specified metamodel (metamodel level only)	No	Class diagram (metamodel layer)	Yes	No	Tool implementation
[23]	Using Alloy language as a formal modeling language	No	Models specified in Alloy	Yes	Yes	Using Alloy Analyzer

3. Methodology

In this section, we outline the research methodology used to ensure consistency across basic UML models. Initially, we study the UML metamodel, which is provided by the Object Management Group (OMG), Milford, MA, USA. Then, we derive diagram-specific metamodels for the basic UML models independently: use case, class, activity, sequence, and state. In the proposed metamodels, only essential elements relevant to the purpose of this study are represented. Since elements of different metamodels are interrelated, we introduce a comprehensive metamodel that highlights cross-model relations among their elements. Furthermore, we define new consistency rules that correspond to cross-model relations in the comprehensive metamodel.

3.1. Proposed Framework

LBF-UML is a framework for detecting inconsistencies of basic UML models, in which a comprehensive metamodel of UML models is constructed, and new consistency rules are defined. The comprehensive metamodel includes entity elements and relationship elements. In practice, there are two types of relationships: structural and semantic. Structural relations define the connections between metamodel elements (M2 level) and appear explicitly as relations between model elements in a user model (M1 level). These kinds of relations are explicitly represented in the UML specification [2]. Semantic relations, on the other hand, capture the correspondences between metamodel elements that govern the consistent interpretation of related model elements and preserve the intended behavior of the system. They describe how information specified in one UML model should be reflected consistently across related models and are not explicitly represented as metamodel elements (M2 level). However, their intended meaning can be inferred from the behavioral descriptions and constraints provided in the UML specification [2]. We introduce additional structural and semantic relationships in the comprehensive metamodel to explicitly capture both intra-model and inter-model (cross-model) correspondences among core metamodel elements. Many cross-model consistency relationships are semantic in nature because they ensure the preservation of the intended system behavior across different UML models rather than merely structural correspondence.

In the proposed framework, relations are classified according to their nature (structural or semantic) and their scope (intra-model or inter-model). Cross-model relations play a central role in consistency management, and we explicitly represent them as first-class elements to support automated analysis and consistency checking. We formally define a new set of consistency rules associated with cross-model relations. Together with the proposed comprehensive metamodel, these rules establish the LBF-UML framework for detecting UML model inconsistencies. The LBF-UML framework is shown in Figure 5 and described in detail below.

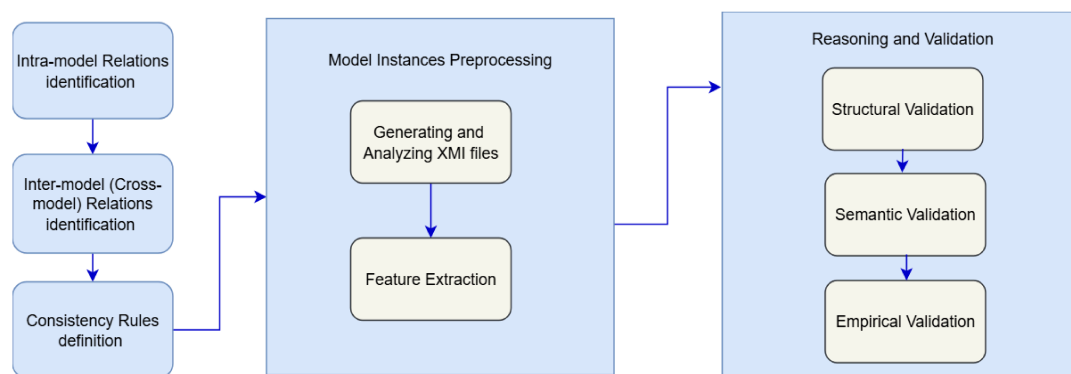


Figure 5. LBF-UML framework.

3.2. Intra-Model Relations

As an initial step, an independent metamodel is constructed for each UML model. The UML has a metamodel that describes its modeling elements and relationships, as defined in the specification document [2]. We are extending the existing metamodel with additional structural and semantic relations, and retaining only those UML elements involved in the proposed consistency framework. Elements unrelated to consistency verification were omitted to keep the metamodel concise and computationally manageable.

The proposed metamodels are defined at the metamodel level (M2) and are templates for user models defined at the model level (M1). The elements in a user model are treated as instances of corresponding metamodel elements and have the structural and semantic

relationships defined at the metamodel level. Thus, consistency rules defined on metamodel elements can be systematically applied to use model elements. This allows for uniform consistency verification of different UML diagrams and ensures the traceability between metamodels and instances of models generated by the user. The proposed metamodels use standard UML multiplicity notations, including exact cardinalities (e.g., 1 or other fixed numbers), 0..1 (zero or one), * (zero or more), 1..* (one or more), and 2..* (two or more) in accordance with the UML specification.

Use Case Diagram. Use cases are a mechanism for capturing the interactions between the system and its users or other systems, using a graphical model and structured text [25]. Use cases are documented using a high-level use case diagram that includes a set of use cases, actors, and relationships. Use cases represent the system’s functionalities as stated in system requirements, while actors represent hardware, software, or people interacting with the system. Use cases and actors are connected through relationships. In addition to these relationships, use cases themselves may be related to each other through relationships, such as include, extend, and generalization. Each use case has a textual description that captures its essential details. The UML metamodel does not cover textual descriptions of use cases; hence, there is no standard metamodel for them [26]. We propose a metamodel of use cases that combines graphical elements with textual descriptions, as a preliminary step toward mapping use case model elements to elements in other UML models (to be discussed in the next sections). We build on a textual use case metamodel proposed in [26]. Figure 6 depicts the proposed metamodel of the use case model, where solid lines denote structural relations while dashed lines denote semantic relations.

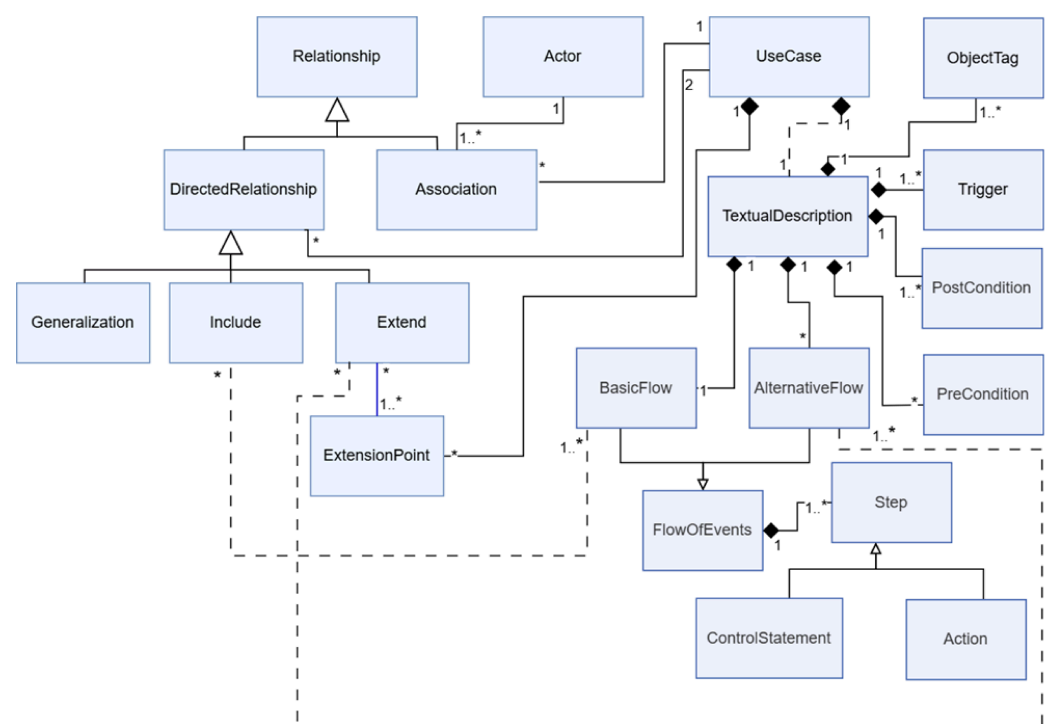


Figure 6. Use case metamodel.

As a preliminary step toward constructing a comprehensive metamodel, we retain the UML elements required for consistency management (*UseCase*, *Actor*, *Association*, *Include*, *Extend*, *ExtensionPoint*, and *Generalization*) and introduce additional elements needed for cross-model mapping.

Each use case is specified with a textual description that identifies its trigger, precondition, postcondition, basic flow, and alternative flow. To reflect this, we introduce a new

semantic relation between the *UseCase* meta class and the *TextualDescription* meta class. The *Include* relationship represents mandatory behavior; therefore, it is associated with the basic flow of the base use case. The *Extend* relationship, on the other hand, reflects a conditional or an optional behavior, and is associated with the alternative flow of the base (extended) use case. Therefore, we introduce a new semantic relation linking the *BasicFlow* meta class to the *Include* meta class, in which the basic flow of a base use case may refer to the included use case behavior in one of its steps. Additionally, a new semantic relation is added between the *AlternativeFlow* meta class and the *Extend* meta class, since an alternative flow of a base use case (extended) may refer to the behavior of the extending use case.

To facilitate mapping use case diagrams to other UML diagrams, we introduce a new meta class, *ObjectTag*, representing domain objects referenced in use case descriptions, and enabling their mapping to elements in other UML models. An *ObjectTag* is attached to the textual description to specify the set of domain objects associated with the use case, which correspond to domain entities represented as classes in a class diagram, and shall be represented as lifelines in the corresponding sequence diagram, according to the proposed consistency rules. For instance, the use case “borrow a book” in a library system involves the following objects: book (representing the item to be borrowed), borrower (representing the user who borrows the book), and loan (representing the entity responsible for the borrowing process), as illustrated in Figure 7. Further analysis may reveal additional objects and add them to the set of object tags.

```

UseCase: Borrow a book
Trigger: The user selects borrow book option
PreCondition: The user has logged in successfully
ObjectTag: borrower, book, loan
.....

```

Figure 7. A textual description of a use case, including an object tag.

Class Diagram. The initial stage of modeling is to study the real world, identify essential objects, and represent them as classes [25]. A class represents a concept or a blueprint for defining a set of similar objects. It consists of attributes and operations. Classes are connected via relationships, which have different types, such as association and generalization. Classes and relationships are the core elements of a class diagram, which captures the system’s structural aspects [25].

Figure 8 shows the class diagram metamodel. To support consistency management, we retain only the UML class diagram elements required by the proposed framework (*Class*, *Property*, *Operation*, *Parameter*, *Association*, *MultiplicityElement*, and *Generalization*) and omit elements that do not participate in cross-model consistency checking. A class consists of attributes (properties) and operations with zero or more parameters. Classes are connected through relationships, which can be classified as either generalization or association. An association has at least two end properties.

Activity Diagram. To represent the system’s dynamic aspects, UML activity diagrams are used. Activity diagrams show the activities in a process, the flow of control from one activity to another, and conditions on the flow [25]. Hence, they are used to model the workflow of a system, business process, or use case. An activity diagram can be seen as an advanced flowchart, as it can represent all aspects of a flowchart. Still, it has extra expressive power: it can represent parallel activities and handle complex decision-making through guards on control flows.

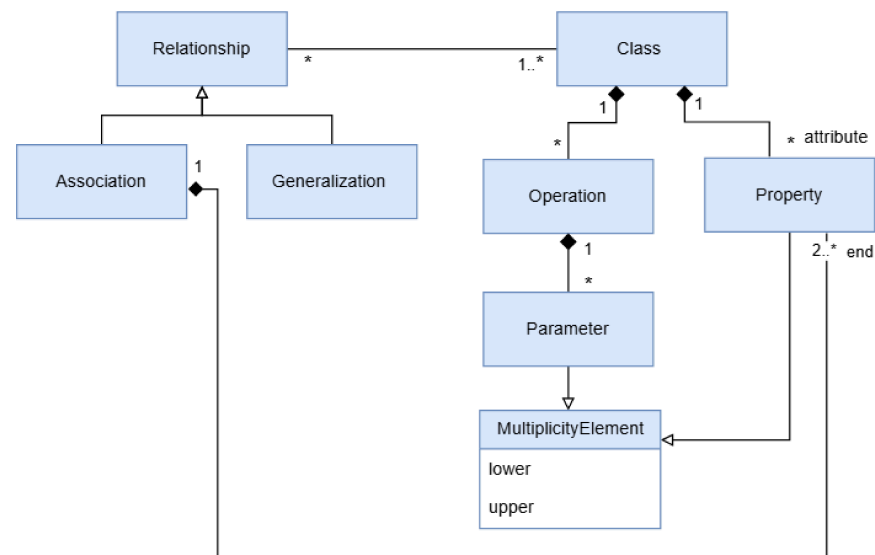


Figure 8. Class diagram metamodel.

Figure 9 depicts the activity diagram metamodel. To support consistency management, we retain only the UML activity diagram elements required by the proposed framework (*ActivityNode*, *ControlNode*, *InitialNode*, *FinalNode*, *DecisionNode*, *MergeNode*, *ActivityEdge*, and *Guard*) and omit elements that do not participate in cross-model consistency checking. An activity metamodel consists mainly of two elements: a node and an activity edge. A node is either an activity node or a control node. A control node can be classified into an initial node, a final node, a decision node, or a merge node. An initial node has one outgoing activity edge, whereas a final node has one or more incoming activity edges. A decision node has one incoming activity edge and two or more outgoing activity edges, while a merge node has two or more incoming activity edges and one outgoing activity edge. An activity edge may have a guard to determine which path is taken when a decision node is reached.

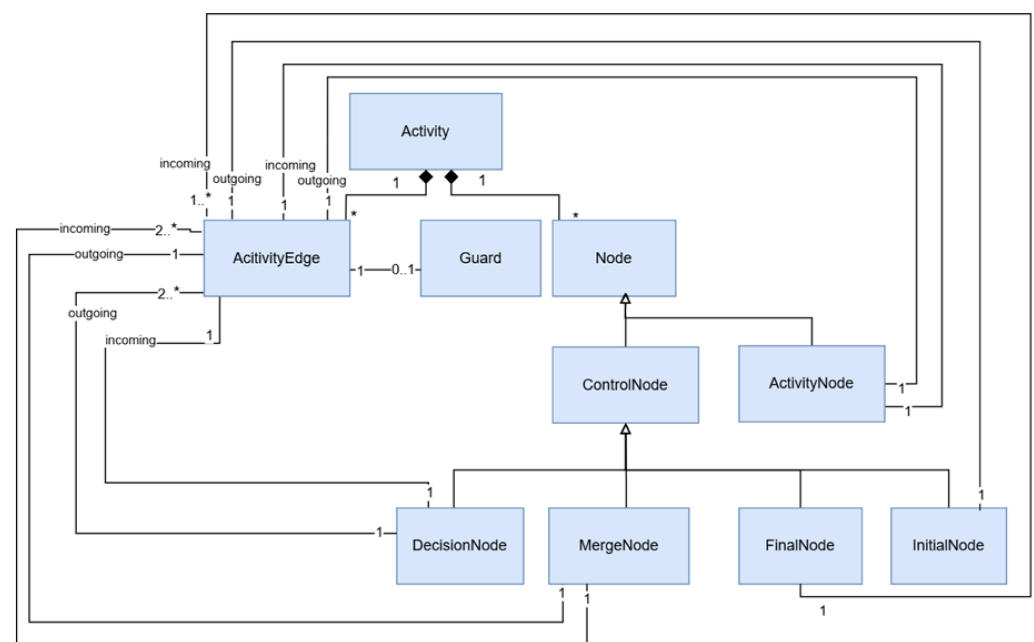


Figure 9. Activity diagram metamodel.

Sequence diagram. UML Sequence diagrams are primarily used to model interactions between actors and objects in a system, as well as interactions among objects [25]. A sequence diagram depicts the sequence of messages exchanged between system components [2].

A sequence diagram can be modeled at different levels of abstraction. At a high level, it can be used to illustrate the sequence of exchanged messages between the actors and the system as a black box; this is referred to as a System Sequence Diagram (SSD). At a more detailed level, it can show the internal objects participating in message exchange, which is known as a design sequence diagram.

Figure 10 represents our metamodel of the sequence diagram. To support consistency management, we retain only the UML sequence diagram elements required by the proposed framework (*Interaction*, *Lifeline*, *Message*, *InteractionFragment*, *CombinedFragment*, *InteractionOperand*, *Guard*, and *InteractionUse*). An interaction consists of a lifeline, a message, and an interaction fragment that captures the execution flow. Each message has a message sort (synchronous or asynchronous), a sender lifeline, and a receiver lifeline. We introduce a new meta class, *Order*, to explicitly define message ordering. UML provides ordering using occurrence specifications and general ordering relationships, whereas this sequence metamodel offers a simplified representation, where we explicitly define message ordering required by the proposed consistency rules.

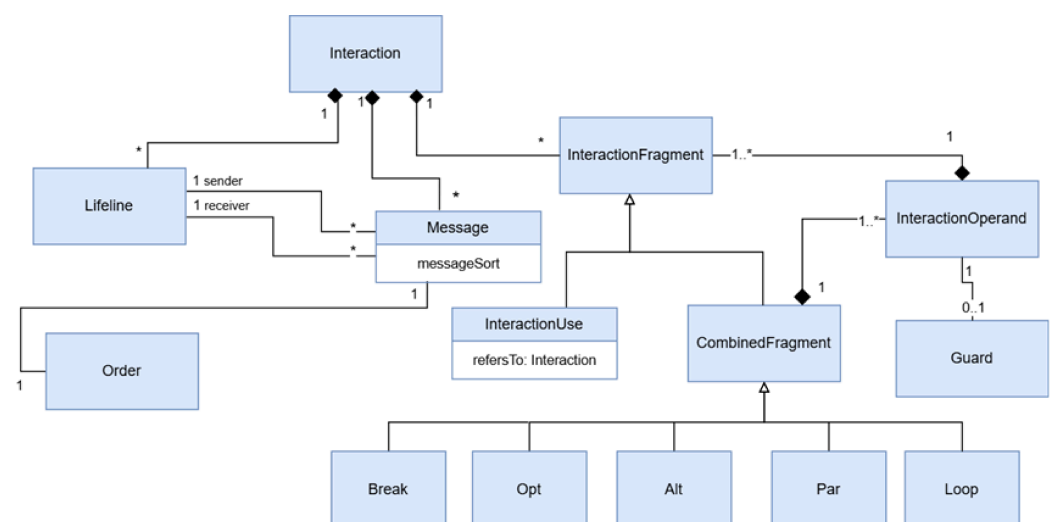


Figure 10. Sequence diagram metamodel.

Interaction fragments represent a unit of behavior in a sequence diagram. To group interactions, sequence diagrams use combined fragments (also referred to as frames in graphical notations). Combined fragments contain interaction operands, which are fragments of interactions associated with guards. An “alt” frame denotes alternatives and has two interaction operands, the first one is performed when the value of the guard is determined to be true, and the second one is performed when the value of the guard is determined to be false, as in the case of an if-then-else structure. An “opt” frame, on the other hand, is used to show options and has only one interaction operand, like an if-then structure. When an interaction involves repetition, a “loop” frame can be used, requiring a guard to specify the number of repetitions. A “par” frame is used to represent concurrent interaction, while a “break” frame is used to represent an interruption in the primary interaction. Interaction use allows the reuse of interactions; i.e., an interaction can reference another interaction, which is normally represented on sequence diagrams with a “ref” frame.

State Diagram. A state machine or a state diagram, as called in the UML, is a behavior diagram that illustrates the discrete behavior of a part of a designed system through finite state transitions [25]. The proper sequence of messages/operations can also be defined using state diagrams. UML defines two kinds of state machine diagrams: behavior state machines, which describe the lifecycle of an object, and protocol state machines, which define the valid sequences of behavioral features to be invoked, such as operations [2].

Figure 11 depicts the state diagram metamodel. As an initial step toward consistency management, we retain only the UML state diagram elements required by the proposed framework (*State*, *Transition*, *Trigger*, *Guard*, and *Event*) and omit elements that do not participate in cross-model consistency checking. A state diagram consists of transitions and states. A state represents the invariant conditions or circumstances of an object, belonging to a class, at an instant in time. The invariant may represent a static or dynamic situation. An event is an occurrence at a specific point in time that may trigger a transition. A transition is a change from one state, the source state, to another state, the target state.

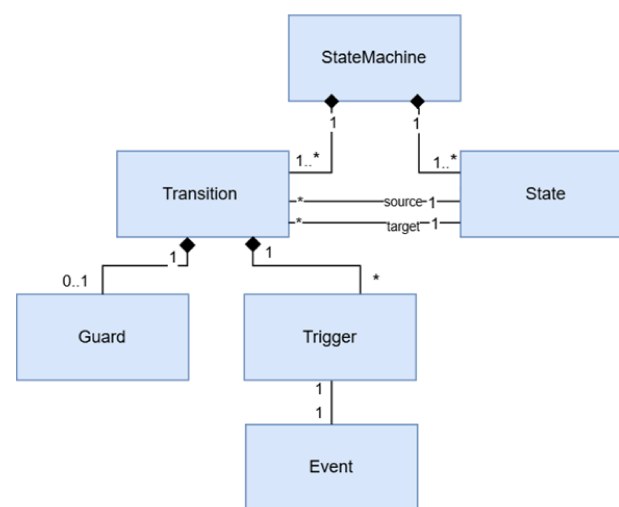


Figure 11. State diagram metamodel.

3.3. Inter-Model (Cross-Model) Relations

A subsequent step in building independent metamodels is to link them to form a single, comprehensive metamodel. This supports model validation, traceability, and consistency management. The comprehensive metamodel defines cross-model relations among different metamodel elements. Our comprehensive metamodel serves as a bridge for defining new consistency rules and illustrates how elements in one metamodel correspond to elements in another.

Figure 12 illustrates the cross-relations between the different UML models. A use case involves a set of classes, since the textual description of a use case encompasses different objects that are instances of classes in a class diagram. A use case behavior is detailed in an activity diagram and precisely specified in a sequence diagram, which in turn illustrates the class instances participating in the use case scenario. A state machine models the behavior of an object that is an instance of a class in the class diagram. The defined cross-model relations include both structural correspondences (e.g., between use case object tags and class elements) and semantic correspondences (e.g., between a use case trigger and a sequence message, and between an include relationship in a use case diagram and an interaction use in a sequence diagram). The comprehensive metamodel is presented in detail in Appendix A.

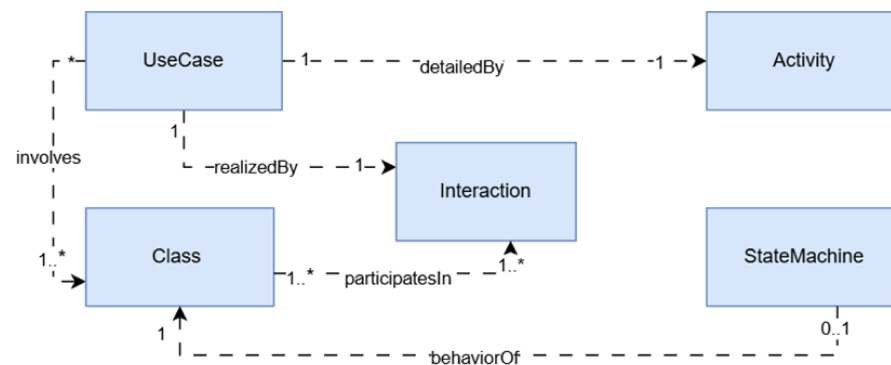


Figure 12. The comprehensive metamodel.

3.4. Consistency Rules Definition

Consistency rules to be enforced in the consistency management process are defined. These rules are specified using first-order logic as a knowledge representation method and are subsequently translated into Alloy specifications to enable automated consistency validation, as discussed in the following sections.

Seven consistency rules are defined as part of the development of the LBF-UML framework rather than adopted from an existing catalog of UML consistency constraints. The seven rules were defined to cover both structural and behavioral consistency aspects, capturing the principal traceability relationships through which requirements are propagated across multiple UML views. They were designed to demonstrate how key cross-model traceability relationships can be formally specified and automatically verified within the proposed framework. Rules R1 and R2 ensure that domain entities identified during requirements analysis are consistently represented in structural and interaction models.

Rules R3, R5, and R6 verify that behavioral information expressed in use case descriptions, including triggers and include/extend relationships, is consistently reflected in sequence diagrams. Rule R4 ensures that alternative execution paths specified in use cases are preserved in activity models, while Rule R7 establishes consistency between use case triggers and state-machine behavior. Together, these rules cover both structural consistency (R1–R2) and behavioral consistency (R3–R7) and address relationships whose violations may lead to inconsistent interpretation of system functionality, incomplete requirements traceability, or behavioral mismatches across UML models. The proposed rules are created for the UML diagrams that are addressed in this framework: use case, class, activity, sequence, and state diagrams. These rules are applicable if the elements of the model and the cross-model mappings are available. The framework is extensible and can accommodate additional metamodel elements, UML diagram types, and consistency rules.

Below, we present the newly defined consistency rules for basic UML models. Rules expressed using the word “must” denote mandatory rules, whereas rules specified using the word “should” are recommended to enhance consistency. Let u , $u1$, and $u2$ denote use cases, o an object, t a textual description, c a class, s , $s1$, and $s2$ sequence diagrams, l a lifeline, tr a trigger, f an alternative flow, a an activity diagram, d a decision node, fr , $f1$, and $f2$ frames, st a state diagram, and e an event, then the consistency rules are defined as follows:

Rule 1.

$$\forall u \forall o \forall t ((\text{UseCase}(u) \wedge \text{Object}(o) \wedge \text{TextualDescription}(u,t) \wedge \text{AppearsIn}(t,o)) \rightarrow \exists c (\text{Class}(c) \wedge \text{InstanceOf}(o,c)))$$

Every object o that appears in a textual description t of a use case u must be an instance of a class c in the class diagram. This rule helps to ensure consistency between the requirements captured in use case descriptions and the structural elements represented in

class diagrams. A violation occurs when a domain object, described in a use case, does not have a corresponding class representation.

Rule 2.

$$\forall u \forall o \forall t (\text{UseCase}(u) \wedge \text{Object}(o) \wedge \text{TextualDescription}(u,t) \wedge \text{AppearsIn}(t,o) \rightarrow \exists s \exists l (\text{SequenceDiagram}(s) \wedge \text{Lifeline}(l) \wedge \text{CorrespondsTo}(s,u) \wedge \text{Represents}(l,o)))$$

Every object o appearing in a textual description t of a use case u , and hence an instance of a class c in the class diagram (according to Rule 1), must appear in the corresponding sequence diagram s of that use case as a lifeline l . This rule helps maintain consistency between the requirements that are captured in the use case descriptions and the interactions modeled in sequence diagrams. A violation occurs when there is no domain object in a use case description that has a corresponding lifeline in the associated sequence diagram.

Rule 3.

$$\forall u \forall t (\text{UseCase}(u) \wedge \text{TextualDescription}(u,t) \rightarrow \exists tr \exists s (\text{Trigger}(tr) \wedge \text{AppearsIn}(t,tr) \wedge \text{SequenceDiagram}(s) \wedge \text{CorrespondsTo}(s,u) \wedge \text{FirstMessage}(s,tr)))$$

For every use case u with a textual description t , there exists a trigger tr of the use case u where this trigger must be the first message in the corresponding sequence diagram s of that use case. The purpose of this rule is to ensure that the interaction behavior represented in the sequence diagram is consistent with the triggering condition specified in the use case description. A violation is observed when the trigger of the use case does not come first in the message flow of the associated sequence diagram.

Rule 4.

$$\forall u \forall t \forall f (\text{UseCase}(u) \wedge \text{TextualDescription}(u,t) \wedge \text{HasAlternativeFlow}(t,f) \rightarrow \exists a \exists d (\text{ActivityDiagram}(a) \wedge \text{CorrespondsTo}(a,u) \wedge \text{DecisionNode}(d) \wedge \text{Contains}(a,d) \wedge \text{Represents}(d,f)))$$

If a use case u has an alternative flow f in its textual description t , then the corresponding activity diagram a of use case u should include a decision node d . This rule ensures the alternative flows of a use case description are consistent with the corresponding activity diagram. Decision nodes provide the control-flow mechanism required to model alternative behaviors. If an activity diagram associated with a use case containing an alternative flow does not include a decision node, the model may need further review.

Rule 5.

$$\forall u1 \forall u2 (\text{UseCase}(u1) \wedge \text{UseCase}(u2) \wedge \text{Include}(u1,u2) \rightarrow \exists s1 \exists s2 \exists fr (\text{SequenceDiagram}(s1) \wedge \text{SequenceDiagram}(s2) \wedge \text{CorrespondsTo}(s1,u1) \wedge \text{CorrespondsTo}(s2,u2) \wedge \text{Frame}(fr) \wedge \text{Type}(f,ref) \wedge \text{Contains}(s1,fr) \wedge \text{Reference}(fr,s2)))$$

If a base use case $u1$ has an include relationship with another use case $u2$ ($u1$ includes $u2$), then in the corresponding sequence diagram $s1$ of the use case $u1$, there should be a frame fr of type ref that refers to the sequence diagram $s2$ that corresponds to the included use case $u2$. When an include relationship is defined between use cases, this rule ensures capturing the inclusion in the corresponding sequence diagrams. When the behavior of the included use case is not referenced in the sequence diagram of the base use case, the model needs further review.

Rule 6.

$$\forall u1 \forall u2 (\text{UseCase}(u1) \wedge \text{UseCase}(u2) \wedge \text{Extend}(u2,u1) \rightarrow \exists s1 \exists s2 \exists f1 \exists f2 (\text{SequenceDiagram}(s1) \wedge \text{SequenceDiagram}(s2) \wedge \text{CorrespondsTo}(s1,u1) \wedge \text{CorrespondsTo}(s2,u2) \wedge \text{Frame}(f1) \wedge (\text{Type}(f1,opt) \vee \text{Type}(f1,alt) \vee \text{Type}(f1,break)) \wedge \text{Frame}(f2) \wedge \text{Type}(f2,ref) \wedge \text{NestedFrames}(f1,f2) \wedge \text{Reference}(f2,s2)))$$

If a base use case $u1$ (extended use case) has an extend relationship with an extending use case $u2$ ($u2$ extends $u1$), then in the corresponding sequence diagram $s1$ of the base

use case $u1$, there should be a frame $f2$ whose type is a *ref* frame, referring to the sequence diagram $s2$ that corresponds to the extending use case $u2$, nested inside $f1$ frame whose type is *opt*, *alt* or a *break*, to denote an optional or an exceptional behavior. When an extended relationship is defined between use cases, this rule ensures that the extending behavior is captured consistently in the corresponding sequence diagrams. When the behavior of the extending use case is not referenced in the sequence diagram of the base use case through an appropriate fragment, the model needs further investigation.

Rule 7.

$$\forall u \forall t \forall tr (\text{UseCase}(u) \wedge \text{Trigger}(tr) \wedge \text{TextualDescription}(u,t) \wedge \text{AppearsIn}(t,tr) \wedge \exists o \exists c \exists st (\text{Object}(o) \wedge \text{AppearsIn}(t,o) \wedge \text{Class}(c) \wedge \text{InstanceOf}(o,c) \wedge \text{StateDiagram}(st) \wedge \text{Owns}(c,st)) \rightarrow \exists e \exists st (\text{Event}(e) \wedge \text{Represents}(e,tr) \wedge \text{BelongsTo}(e,st)))$$

For a use case u with a textual description t containing a trigger tr , if at least one tagged object o belongs to a class c that owns a state diagram st , then the trigger tr must correspond to an event e in at least one of the associated state diagrams. This rule ensures that the trigger of a use case is represented consistently as an event in at least one state diagram associated with the tagged objects. A violation occurs when a use case contains tagged objects whose corresponding classes own state diagrams, but the trigger identified in the use case description is not represented as a corresponding event in any of those state diagrams.

3.5. Model Preprocessing

UML user models are converted to their corresponding XMI representations to facilitate consistency checking. XMI, which stands for XML Metadata Interchange, is a standard format for exchanging metadata information, especially UML models, between different tools and platforms. The resulting XMI files are parsed automatically to extract structural and semantic information about UML models. Essential UML model elements that should be checked are defined and located, including classes, attributes, methods, and associations, etc. These elements and their relationships are identified and mapped according to the proposed metamodels and are subsequently used for automated consistency verification against the defined consistency rules.

3.6. Reasoning and Validation

The defined metamodels, cross-model relations, and consistency rules are formally expressed using Alloy Analyzer, which is a formal reasoning tool. Alloy Analyzer checks the internal consistency of the specification (referred to as a model in Alloy). If the specification is found to be consistent, an instance is generated. Otherwise, Alloy fails to create a valid instance.

To validate the LBF-UML framework, we conduct validation at three levels: structural, semantic, and empirical validation.

- **Structural validation.** This level of validation includes checking the comprehensive metamodel for internal consistency. To achieve this, we employ Alloy Analyzer for validation at two levels: first, every UML metamodel is validated independently; second, the comprehensive metamodel containing cross-model relations is validated.
- **Semantic validation.** This validation ensures that consistency rules are meaningful and correspond to the intended meaning. Instances generated by Alloy Analyzer are inspected to ensure they not only satisfy all constraints but also reflect the intended meaning. To achieve this, Instances generated by Alloy have been examined by three domain experts to ensure their correctness.
- **Empirical validation.** This is an experimental evaluation of the framework, using different UML user models. The defined consistency rules are applied to these UML models

to detect potential inconsistencies. This validation aims to assess the applicability of our approach to real-world scenarios.

4. Implementation

The implementation of the LBF-UML framework relied primarily on three tools: Modelio 5.4.01, Python 3.12, and Alloy Analyzer 6.2.0, as illustrated in Figure 13. Modelio was used as a graphical modeling environment to create UML user models, as depicted in Figure 14, and to export them as XMI files.

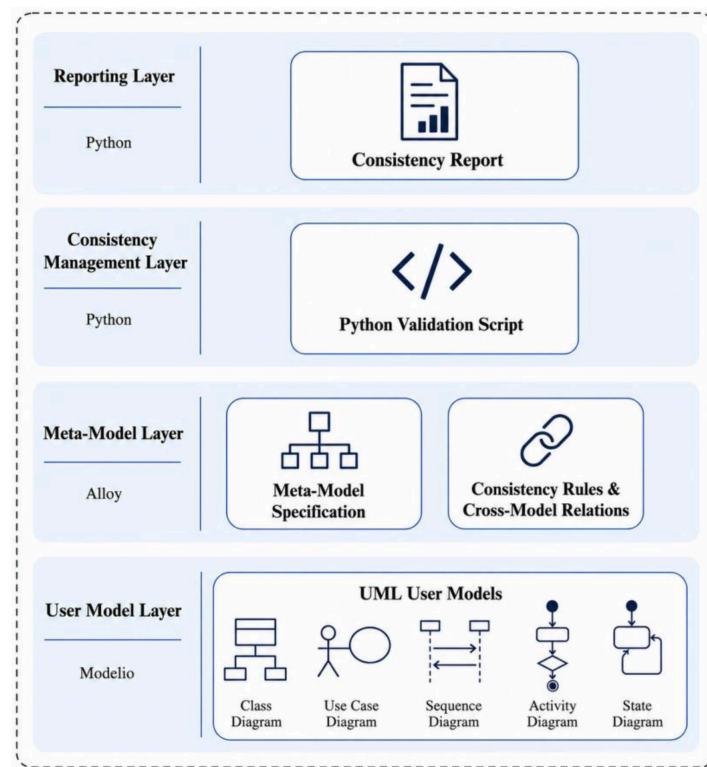


Figure 13. LBF-UML architecture.

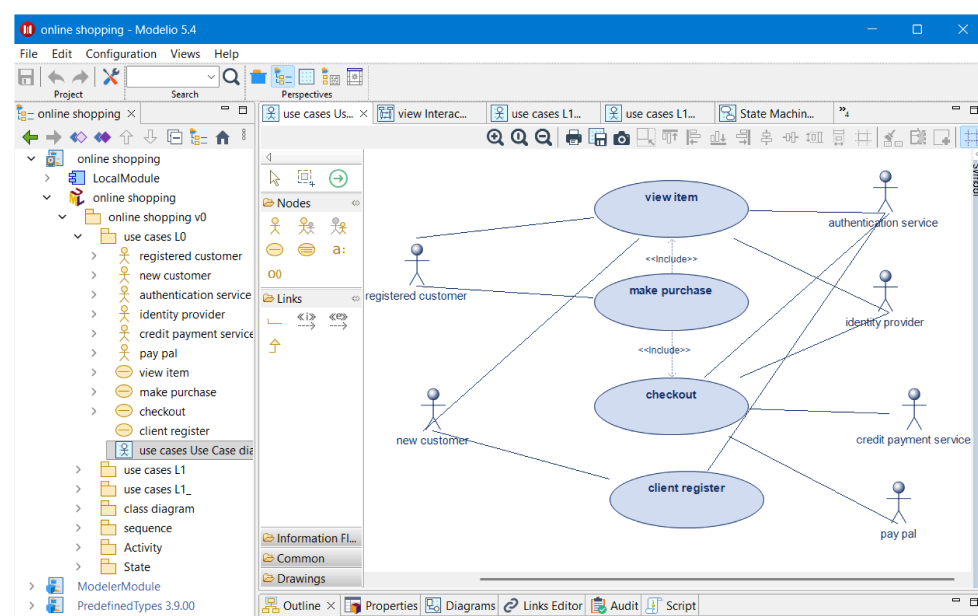


Figure 14. Graphical modeling performed using the Modelio tool.

A sample of an XMI file is shown in Figure 15. Modelio adheres to UML standards and is therefore considered a robust tool whose output was used in subsequent stages of consistency management. The newly introduced meta class *ObjectTag* was added to use case textual description, as illustrated in Figure 16. The implementation currently relies on manual assignment of *ObjectTag* instances by the modeler to identify domain objects that are referenced in the textual description of a use case. No manual manipulation of XMI files is required for the framework. *ObjectTag* instances are introduced at the modeling phase in Modelio, as part of the use case description. The annotations are then automatically exported as part of the generated XMI. The effort required will vary according to the description of the use case and the number of domain entities referenced.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <uml:Model xmi:version="2.1" xmlns:xmi="http://schema.omg.org/spec/XMI/2.1" xmlns:uml="http://www.eclipse.org/uml2/1.1.0/UML">
3   <eAnnotations xmi:id="_r_YaIbpAEfC80dEE8_LW1Q" source="Objing">
4     <contents xmi:type="uml:Property" xmi:id="_r_YaIrpAEfC80dEE8_LW1Q" name="exporterVersion">
5       <defaultValue xmi:type="uml:LiteralString" xmi:id="_r_YaI7pAEfC80dEE8_LW1Q" value="3.0.0"/>
6     </contents>
7   </eAnnotations>
8   <packagedElement xmi:type="uml:Package" xmi:id="_r_YaJLpAEfC80dEE8_LW1Q" name="Package">
9     <packagedElement xmi:type="uml:Class" xmi:id="_r_YaJbpAEfC80dEE8_LW1Q" name="Class">
10      <ownedAttribute xmi:id="_r_YaJrpAEfC80dEE8_LW1Q" name="class1" visibility="public" type="_r_YaMbpAEfC80dEE8_LW1Q">
11        <lowerValue xmi:type="uml:LiteralInteger" xmi:id="_r_YaJ7pAEfC80dEE8_LW1Q"/>
12      </ownedAttribute>
13      <ownedAttribute xmi:id="_r_YaKlpAEfC80dEE8_LW1Q" name="class2" visibility="public" type="_r_YaMrpAEfC80dEE8_LW1Q">
14        <lowerValue xmi:type="uml:LiteralInteger" xmi:id="_r_YaKbpAEfC80dEE8_LW1Q"/>
15      </ownedAttribute>
16      <ownedAttribute xmi:id="_r_YaKrpAEfC80dEE8_LW1Q" name="attr1" visibility="public" isUnique="false">
17        <type xmi:type="uml:PrimitiveType" href="pathmap://UML_LIBRARIES/UMLPrimitiveTypes.library.uml#String"/>

```

Figure 15. An XMI file generated by Modelio.

Use Case	Exceptions	Conditions	Constraints	ObjectTag
Description	Preconditions: User must be registered in the system. Main Flow: Borrower selects a book, system checks availability, creates a loan record. Postconditions: Book is marked as loaned. ObjectTag: book			

Figure 16. *ObjectTag* is added to the use case textual description.

At the metamodel layer, we employed Alloy to formally specify and analyze metamodels, cross-model relations, and consistency rules. An Alloy model consists of signatures, assertions, facts, functions, predicates, and commands [14]. Each set of elements is represented by a signature declaration, specified using the keyword *sig*, which is analogous to a unary predicate in logic. A signature may contain declarations of relations, which are considered as fields within the signature and may contain keywords of multiplicity inside them. A signature may be a subset of another signature using the keyword *extends*, representing specialization through subset inclusion. Assertions, analogous to theorems in logic,

are statements in which the keyword *assert* is used to declare properties that are expected to be true. A fact, on the other hand, is a constraint that is presumed to be always true and is denoted by the keyword *fact*, like an axiom in logic. Reusable expressions can be grouped into functions using the *fun* keyword, whereas reusable constraints can be grouped into predicates using the keyword *pred*. Alloy uses the commands *run* and *check* to perform a specific analysis. The command *run* finds an instance satisfying a model, and the command *check* finds a counterexample to assertions, i.e., an instance that falsifies assertions.

Every metamodel was modeled as an Alloy model, with all metamodel elements and relations (structural and semantic) represented as signatures and relations. Two fundamental elements of our framework, which are cross-model relations and consistency rules, namely at the metamodel level, were represented as Alloy facts and predicates. Figure 17 shows an Alloy textual use case metamodel. We employed Alloy Analyzer 6.2.0, which is a tool with a SAT solver that analyzes and validates Alloy models. Alloy Analyzer can produce a diagrammatic depiction of metamodels and of valid model instances that adhere to them. This graphical illustration is a graph, whose nodes represent elements and edges represent relations between them. A graphical metamodel of the use case model is shown in Figure 18, and a valid model instance is shown in Figure 19. We employed the *util/ordering* library, a standard Alloy module, to model ordering among model elements, as some UML models, such as sequence diagrams, impose ordering on exchanged messages.

```
// Module declaration
module UC_new

sig UseCase {
    textualDesc: one TextualDescription, // one TextualDescription
    extensionPoints: set ExtensionPoint , // 0..* extension points
    include: set UseCase
}

sig Actor {}

abstract sig URelationship {}

sig Association extends URelationship {
    useCaseEnd: one UseCase,
    actorEnd: one Actor
}

abstract sig DirectedRelationship extends URelationship {
    source: one UseCase,
    target: one UseCase
}

sig Generalization extends DirectedRelationship {}
sig Include extends DirectedRelationship {
    ownerFlow: some BasicFlow //1..*
```

Figure 17. An alloy textual specification of the use case metamodel.

We used PyCharm 2024.2.4 to run a Python script that processes XMI files exported from Modelio. The script was used to parse and analyze XMI files to detect core UML model elements and their relationships according to the proposed metamodels. The script implemented the defined consistency rules, which were previously validated using Alloy, and applied these rules to the XMI files exported by Modelio to check the consistency of user models. The result was a consistency report indicating whether the UML user models

were consistent, and, if not, an explanation of the inconsistency type was displayed, as depicted in Figure 20.

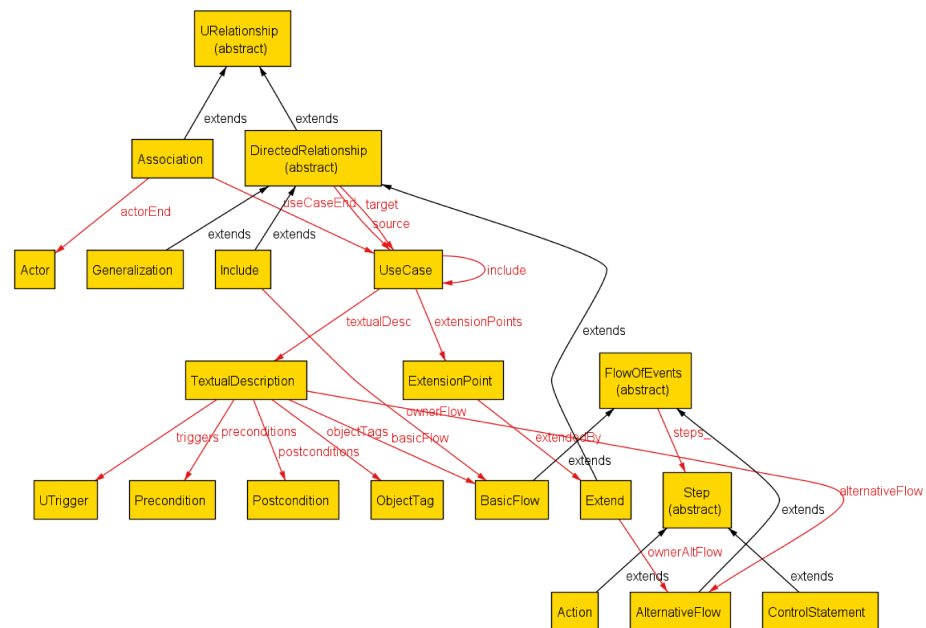


Figure 18. Use case graphical metamodel generated by Alloy Analyzer.

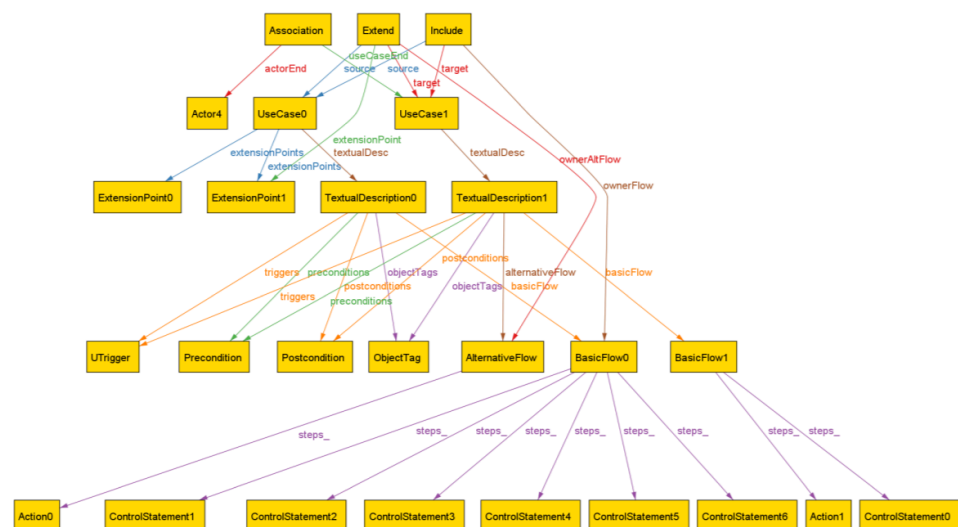


Figure 19. A valid instance of the use case metamodel.

```

=== 🚩 RULE 2 : UC_SEQ_ObjectTags ===
[["UC-SEQ ObjectTag] Use Case 'view item' requires objectTag 'product', but Interaction
'__8AzQgWYefGAS7nvoHpUTg' does not represent it (checked lifeline names and represented
types)."]
|
=== 🚩 RULE 3 : UC_SEQ_Trigger ===
No violation detected

=== 🚩 RULE 4 : UC_AD_AlternativeFlowDecision ===
No violation detected

```

Figure 20. A consistency report generated by the Python script.

5. Experimental Evaluation

This section evaluates the proposed LBF-UML framework in terms of its ability to detect consistency violations across multiple UML diagram types and its applicability in model verification scenarios. The evaluation includes a mutation-based consistency detection experiment and performance analysis using different UML model versions containing varied inconsistencies.

5.1. Experimental Setup

The evaluation was conducted using the implementation environment presented in Section 4, which integrates Modelio, Python, and Alloy Analyzer.

The LBF-UML framework was evaluated using two case studies from different application domains: an online shopping system and a hospital management system, both available on UML-diagrams.org [27]. The case studies included varied, interconnected UML diagrams that model the main functionalities of the systems, for instance, product browsing, order management, and payment processing in the online shopping system, as well as patient registration, appointment scheduling, and medical record management in the hospital management system. Different model versions containing varied consistency violations were analyzed to evaluate the effectiveness of the proposed consistency checking rules. In addition, rule interaction and execution performance were examined to assess the practical applicability of the framework. Table 2 provides an overview of the case studies' statistics. To ensure comprehensive rule coverage, selected UML models were extended with additional model elements.

Table 2. Case studies models statistics.

Element Type	Online Shopping System	Hospital Management System
Classes	8	10
Attributes	28	8
Operations	38	7
Associations	10	4
Actors	6	3
Use Cases	19	9
Actor-use case association	17	8
Include	10	2
Extend	5	2
Generalization	5	2
Sequence Diagrams (Interactions)	5	1
Lifelines	24	4
Messages	40	8
Interaction Uses	3	0
Activity Diagrams	2	1
Activity Nodes	19	5
Control Flows	19	8
State machines	1	1
States	8	5
Transitions	16	7

The primary evaluation dataset is the online shopping system; it was employed for consistency detection and performance analysis. The second dataset from the healthcare domain was analyzed to validate the proposed framework's applicability across different domains.

5.2. Consistency Detection Evaluation

A mutation-based evaluation was used to assess the effectiveness of the proposed consistency checking rules on the two case studies described in Section 5.1. As an initial step in the mutation-based evaluation, a consistent version of the UML models that satisfied all previously specified consistency rules was designated as the baseline version and used as the reference model. Mutated versions were then generated by introducing multiple simultaneous inconsistencies across different UML artifacts (e.g., use case, class, sequence, activity, and state models). Tables 3 and 4 present the baseline and mutated versions for the two case studies, along with the corresponding violated consistency rules, where 0 indicates no violation, 1 indicates a violation, and N/A indicates that the rule is not applicable. A violation was considered to occur when at least one consistency rule was not satisfied. The experimental results demonstrated that the framework successfully detected the injected inconsistencies across all evaluated versions of both case studies. Tables 5 and 6 provide detailed descriptions of the mutation scenarios used in the evaluation.

Table 3. Consistency rule violations across different versions of the online shopping system.

Version	Rules						
	R1	R2	R3	R4	R5	R6	R7
version 0	0	0	0	0	0	0	0
version 1	1	1	0	1	0	0	0
version 2	1	1	1	0	0	1	0
version 3	0	0	0	1	1	0	1
version 4	0	0	1	0	1	1	0

Table 4. Consistency rule violations across different versions of the hospital management system.

Version	Rules						
	R1	R2	R3	R4	R5	R6	R7
version 0	0	0	0	0	0	0	0
version 1	1	1	1	1	N/A	N/A	1
version 2	0	0	1	N/A	1	1	1

Table 5. Mutation description of different versions of the online shopping system.

Version	Mutation Description	Violated Rules
version 1	- Object tag “item” in “view item” use case does not belong to any class.	R1, R2, R4
	- The sequence diagram for “view item” does not include “item” as a lifeline.	
	- Use case “view item” has alternative flows, but its activity diagram contains no decision node.	
version 2	- Object tag “item” in “view item” use case does not belong to any class.	R1, R2, R3, R6
	- The sequence diagram for “view item” does not include “item” as a lifeline.	
	- Use case “view item” has trigger “view cart”, but it is not the first message in the sequence diagram for “view item”.	
	- Use case “view item” is extended by “add to shopping cart” use case, but the sequence diagram of “view item” does not reference “add to shopping cart”.	

Table 5. Cont.

Version	Mutation Description	Violated Rules
version 3	- Use case “view item” has alternative flows, but its activity diagram contains no decision node. - Use case “make purchase” includes “view item”, but the corresponding sequence diagram of “make purchase” does not reference “view item”. - A trigger “select register” exists, but no event in the state machine of any tagged object matches the trigger.	R4, R5, R7
version 4	- Use case “view item” has trigger “view cart”, but it is not the first message in the sequence diagram for “view item”. - Use case “make purchase” includes “view item”, but the corresponding sequence diagram of “make purchase” does not reference “view item”. - Use case “view item” is extended by “add to shopping cart” use case, but the sequence diagram of “view item” does not reference “add to shopping cart”.	R3, R5, R6

Table 6. Mutation description of different versions of the hospital management system.

Version	Mutation Description	Violated Rules
version 1	- Object tag “appointment” in “register patient” use case does not belong to any class. - The sequence diagram for “register patient” does not include “appointment” as a lifeline. - Use Case “register patient” has a trigger “add patient”, but it is not the first message in the “register patient” interaction. - Use case “register patient” has alternative flows, but its activity diagram contains no decision node. - A trigger “add patient” exists, but does not have a corresponding event in any state machine of the tagged objects.	R1, R2, R3,R4,R7
version 2	- Use Case “register patient” has a trigger “add patient”, but it is not the first message in the “register patient” interaction. - Use case “patient hospital admission” includes “register patient”, but the corresponding sequence diagram of “patient hospital admission” does not reference “register patient”. - Use case “schedule admission” is extended by “register patient” use case, but the sequence diagram of “schedule admission” does not reference “register patient”. - A trigger “add patient” exists, but no event in the state machine of any tagged object matches the trigger.	R3, R5, R6, R7

We analyzed the results in terms of true positives, true negatives, false positives, and false negatives. A true positive was observed when a model contained inconsistencies,

and the proposed framework correctly identified them. A false positive occurred when a model had no inconsistencies, but the framework reported violations. A true negative occurred when the framework correctly found a model consistent without reporting any violations, whereas a false negative occurred when the framework reported no violation, though the model violated some rules. For both the online shopping and hospital management case studies, the framework successfully detected all injected inconsistencies, all of which were true positives, with no false negatives or false positives observed during the evaluation. Since the inconsistencies were introduced through controlled mutations with known ground truth, the results primarily validate the correctness of consistency rules formalization and their execution. Therefore, the evaluation should be interpreted as a validation of the implemented consistency-checking mechanism rather than as a statistical assessment of detection performance. However, further assessment of the framework on inconsistencies found in larger industrial models would give more credibility to its practical usefulness and its generalizability.

5.3. Performance Evaluation

In this section, we evaluate the performance of the LBF-UML framework. First, we demonstrate the performance of the formal validation using Alloy Analyzer, followed by the performance of inconsistency detection using the Python script. All experiments were run on a computer with an Intel Core i7 processor, 16 GB RAM, and running Windows 11. Alloy validation and Python execution times were run in the same execution environment.

5.3.1. Alloy Validation Performance

The Alloy Analyzer was used to validate the consistency of the individual UML metamodels and the integrated metamodel containing the proposed cross-model relationships and consistency constraints. Table 7 presents the validation times obtained for each Alloy specification.

Table 7. Validation time of the Alloy specifications.

Alloy Specification	Verification Time (ms)
Use case metamodel	29
Class diagram metamodel	17
Activity diagram metamodel	12
Sequence diagram metamodel	27
State diagram metamodel	10
Comprehensive metamodel	55

Table 7 shows that all Alloy specifications were validated within 55 ms. The comprehensive metamodel required the longest validation time due to the additional cross-model mappings and consistency constraints. Nevertheless, the observed validation times remained low, indicating that the formal validation stage introduces only minimal computational overhead.

5.3.2. Inconsistency Detection Performance

In this section, we evaluate the performance of the inconsistency detection process using the online shopping case study, which serves as the primary evaluation dataset. The hospital management case study was used to validate the applicability of the proposed framework across domains, whereas performance and scalability analyses were conducted using the larger online shopping dataset. To assess the performance in larger models, synthetic scaled datasets were created based on the original UML models by systematically scaling up and replicating the structures of the models, and increasing their elements

while maintaining the cross-model structural and behavioral relations. Unique identifiers were added to new model elements in all UML models. Table 8 shows the execution time of the Python-based inconsistency detection process. The reported size corresponds to the number of XMI elements produced during model export. As expected, the execution time increased with model size, ranging from 64 ms for the original dataset to 2050 ms (approximately 2.05 s) for the largest scaled dataset. Despite the increase in model size, all experiments completed within approximately 2 s, indicating that the proposed framework remains computationally efficient for the evaluated model sizes.

Table 8. Execution times of the Python-based inconsistency detection process for the online shopping case study.

Case Study	Relative Size	Number of XMI Elements	Execution Time (ms)
Original dataset	1×	672	64
Scaled dataset 1	2×	1344	78
Scaled dataset 2	4×	2688	239
Scaled dataset 3	8×	5376	834
Scaled dataset 4	16×	10,752	2050

5.4. Threats to Validity

Synthetic scaled datasets, based on one case study, were used for the performance evaluation. These datasets were created with the aim of maintaining the structure and behavior of the original model, but further experiments on larger industrial-scale UML repositories from different domains would yield a more thorough evaluation of scalability and generalizability. The mutation scenarios were created to model possible realistic inconsistencies between models, which can happen during the independent evolution of UML artifacts. But there could be other patterns of inconsistency that would not have been considered for the real-world modeling projects. Furthermore, semantic relationships were explicitly defined through metamodel elements and cross-model mappings; therefore, semantic correspondences that are not represented in the metamodel cannot be automatically verified.

6. Results and Discussion

The experiments conducted on both the online shopping and hospital management case studies demonstrated that the LBF-UML framework detected all inconsistencies, and all reported inconsistencies were truly positive. The proposed framework combines structural and behavioral consistency relations across use case, class, sequence, activity, and state diagrams within a unified metamodel. As illustrated in Table 9, the framework supports a broader modeling scope and a wider range of consistency relationships than several related approaches, which often focus on a limited set of UML diagrams or consistency relationships.

Table 9. A comparison of the proposed framework with related work.

Research	Approach	Definition of New Rules	Models Covered	Structural Consistency	Semantic Consistency	Validation/Experiment
[16]	Collecting and documenting 119 consistency rules	No	All	Yes	Yes	No

Table 9. Cont.

Research	Approach	Definition of New Rules	Models Covered	Structural Consistency	Semantic Consistency	Validation/Experiment
[17]	Building a formal representation of the models and using a few existing consistency rules	No	Class, Sequence, and State diagrams	Yes	Yes	Tool implementation
[11]	Building a formal representation of the models and using a few existing consistency rules	No	Class, Sequence, and State diagrams	Yes	Yes	Tool implementation
[10]	Formalizing some existing consistency rules, applying them to 34 UML diagrams	No	Class, Sequence, Activity, State, Use Case, Object, and Composite Structure diagrams	Yes	Yes	Using existing tools (Eclipse Papyrus and Eclipse OCL)
[18]	Integrating a rule-based approach with LLM methodologies, handling consistency between use cases and block diagrams only	Yes	Use case and block diagrams	Yes	No	Tool implementation
[19]	Building a meta-model to check the consistency of the requirements, class diagram, and Java only.	No	Requirements model, class diagram, and code model (Java)	Yes	No	Tool implementation
[9]	Defining consistency checking rules by the user	No	Class, use case, sequence, and state diagrams	Yes	Partial	Tool implementation
[20]	Using composite consistency rules to construct systems' architecture automatically.	New composite rules using existing rules	Class, use case, sequence, and state diagrams	Yes	Yes	Tool implementation
[21]	Integrating Alloy with MOF to detect inconsistencies in any user-specified metamodel (model and metamodel levels)	No	MOF-based models	Yes	No	Tool implementation

Table 9. Cont.

Research	Approach	Definition of New Rules	Models Covered	Structural Consistency	Semantic Consistency	Validation/Experiment
[22]	Integrating OCL with the USE tool to detect inconsistencies in any user-specified metamodel (metamodel level only)	No	Class diagram (metamodel layer)	Yes	No	Tool implementation
[23]	Using Alloy language as a formal modeling language Building a metamodel, defining new consistency rules, and using Alloy as a formal modeling language (model and metamodel levels).	No	Models specified in Alloy	Yes	Yes	Using Alloy Analyzer
LBF-UML framework		Yes	Use case, class, activity, sequence, and state diagrams	Yes	Yes	Using Alloy Analyzer, prototype tool implementation, and case study evaluation

It can be observed that the LBF-UML framework operates at both the metamodel and model levels, where we define new consistency rules and additional structural and semantic relations among UML metamodel elements, and apply these rules to user models to detect inconsistencies. Alloy is used to formally specify the comprehensive metamodel and the rules. We address both structural and behavioral aspects, represented by the following diagrams: use case, class, sequence, activity, and state.

7. Conclusions

Software models representing different viewpoints of the system should remain consistent throughout the development process, as inconsistencies in these models slow development and degrade the quality of the final software product. In this paper, we propose a logic-based approach, LBF-UML, to enforce consistency across basic UML models by detecting potential conflicts and inconsistencies among models produced at different phases. Initially, we defined a comprehensive metamodel that extends the existing UML metamodel with additional structural and semantic relations. Then, we introduced a new set of consistency rules and formally specified them using Alloy. The LBF-UML framework was implemented using Python and Alloy Analyzer and evaluated using two case studies. The results demonstrated that all injected inconsistencies have been correctly detected. This study contributes to UML consistency management in several ways. A comprehensive metamodel was defined to explicitly represent structural and semantic relationships within and across UML models. In addition, a set of consistency rules was introduced and formally specified to support the detection of both structural and behavioral inconsistencies. The proposed LBF-UML framework integrates these metamodels and consistency rules within a unified consistency management approach. Future work is to extend the prototype Python script into a robust software tool, to evaluate the proposed framework on a large-scale industrial case study, and to expand the support for additional consistency rules.

Author Contributions: E.N. was responsible for conceptualization, methodology development, prototype implementation, formal analysis, and original draft preparation. A.H. supervised the research and contributed to conceptualization and methodology refinement, and manuscript review. N.O. contributed to the formal analysis, validation, and review of the logic-related components and the manuscript. S.G. contributed to manuscript review, validation, and general feedback. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The source code, UML models (XMI files), Alloy specifications, and supporting artifacts are available at: <https://github.com/enasaffar/LBF-UML-framework>. (accessed on 14 June 2026).

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A

The complete comprehensive metamodel is presented in this appendix, as shown in Figure A1.

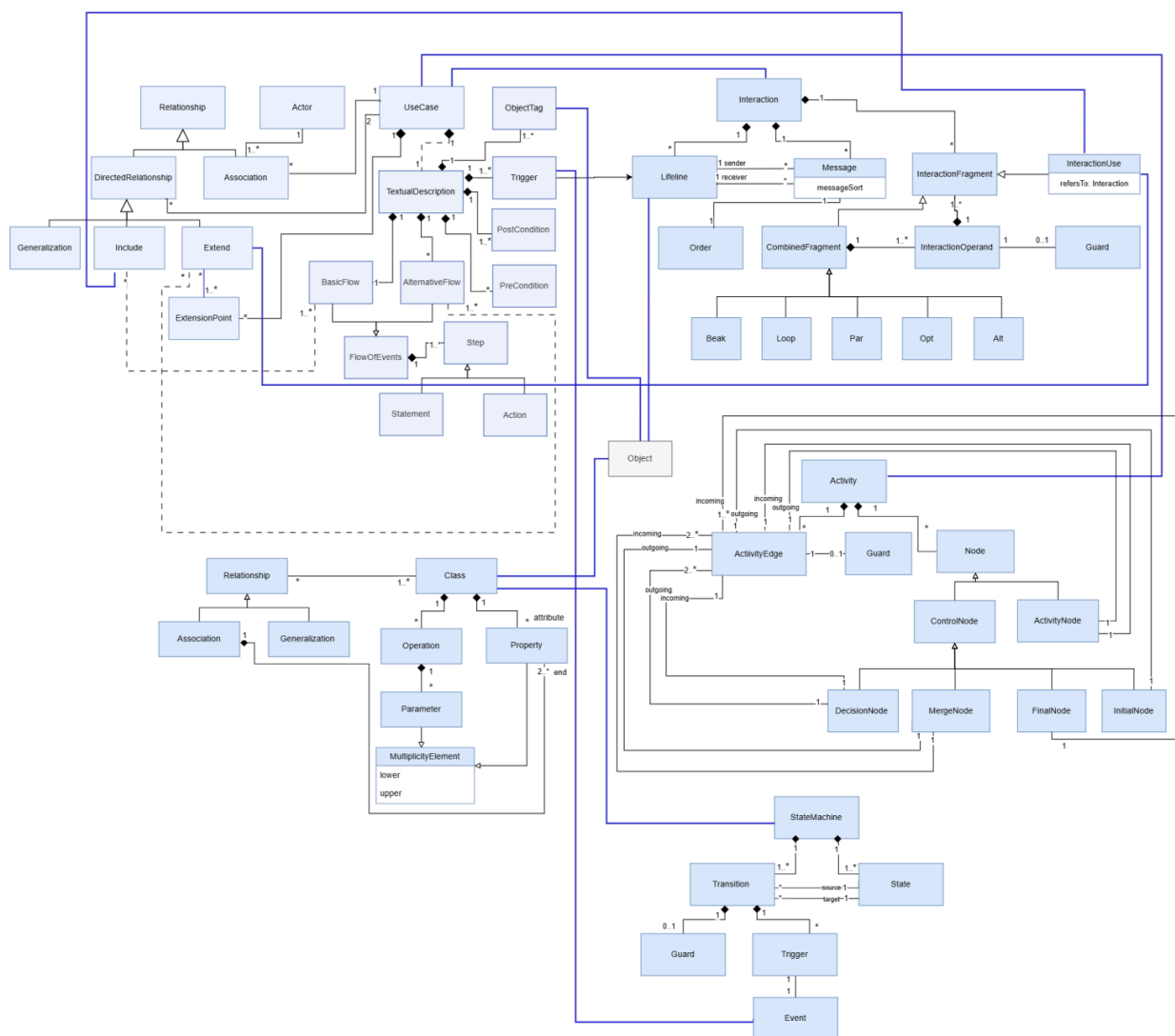


Figure A1. The detailed comprehensive metamodel.

References

- Dennis, A.; Wixom, B.H.; Roth, R.M. *Systems Analysis and Design*, 5th ed.; John Wiley & Sons, Inc.: Hoboken, NJ, USA, 2015.
- Object Management Group (OMG). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*; OMG: Needham, MA, USA, 2017; Available online: <https://www.omg.org/spec/UML/2.5.1/> (accessed on 30 April 2026).
- Schieferdecker, I.K. The Power of Models for Software Engineering. In *The Combined Power of Research, Education, and Dissemination*; Hinchey, M., Steffen, B., Eds.; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2025; Volume 15240. [CrossRef]
- IEEE Computer Society. *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0a*; IEEE Computer Society: Los Alamitos, CA, USA, 2025.
- Allaki, D.; Dahchour, M.; En-Nouaary, A. A New Taxonomy of Inconsistencies in UML Models with Their Detection Methods for Better MDE. *Int. J. Comput. Sci. Appl.* **2015**, *12*, 48–65.
- Usman, M.; Nadeem, A.; Kim, T.-H.; Cho, E.-S. A Survey of Consistency Checking Techniques for UML Models. In *Proceedings of the 2008 Advanced Software Engineering and Its Applications (ASEA 2008), Hainan, China, 13–15 December 2008*; IEEE Computer Society: Los Alamitos, CA, USA, 2008; pp. 57–62. [CrossRef]
- Allaki, D.; Dahchour, M.; En-Nouaary, A. Managing Inconsistencies in UML Models: A Systematic Literature Review. *J. Softw.* **2017**, *12*, 454–471. [CrossRef]
- Spanoudakis, G.; Zisman, A. Inconsistency Management in Software Engineering: Survey and Open Research Issues. In *Handbook of Software Engineering and Knowledge Engineering*; World Scientific Publishing Co.: Singapore, 2001; Volume 1, pp. 329–380.
- Marchezan, L.; Homolka, M.; Blokhin, A.; Assunção, W.K.G.; Herac, E.; Egyed, A. A Tool for Collaborative Consistency Checking During Modeling. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings (MODELS 2024), Linz, Austria, 22–27 September 2024*; ACM: New York, NY, USA, 2024; pp. 655–659. [CrossRef]
- Torre, D.; Labiche, Y.; Genero, M.; Elaasar, M.; Menghi, C. UML Consistency Rules: A Case Study with Open-Source UML Models. In *Proceedings of the 8th International Conference on Formal Methods in Software Engineering (FormalISE '20), Seoul, Republic of Korea, 28–29 June 2020*; ACM: New York, NY, USA, 2020; pp. 130–140. [CrossRef]
- Wen, H.; Wu, J.; Jiang, J.; Tang, G.; Hong, Z. A Formal Approach for Consistency Management in UML Models. *Int. J. Softw. Eng. Knowl. Eng.* **2023**, *33*, 733–763. [CrossRef]
- Object Management Group. *OMG Unified Modeling Language (OMG UML), Infrastructure, Version 2.4.1*; OMG Document Number: Formal/2011-08-05; Object Management Group: Needham, MA, USA, 2011.
- AlloyTools. Alloy Analyzer. Available online: <https://alloytools.org/> (accessed on 30 April 2026).
- Jackson, D. *Software Abstractions: Logic, Language, and Analysis*; MIT Press: Cambridge, MA, USA, 2012.
- Brunel, J.; Chemouil, D.; Cunha, A.; Macedo, N. *Formal Software Design with Alloy 6*; Springer: Cham, Switzerland, 2021.
- Torre, D.; Labiche, Y.; Genero, M.; Elaasar, M. A Systematic Identification of Consistency Rules for UML Diagrams. *J. Syst. Softw.* **2018**, *144*, 121–142. [CrossRef]
- Tang, G.; Jiang, J.; Wen, H. Consistency Analysis of UML Models. In *Proceedings of the 29th International DMS Conference on Visualization and Visual Languages (DMSVIVA 2023), South San Francisco, CA, USA, 29 June–3 July 2023*; KSI Research Inc.: Pittsburgh, PA, USA, 2023; pp. 58–64.
- Sultan, B.; Apvrille, L. AI-Driven Consistency of SysML Diagrams. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS 2024), Linz, Austria, 22–27 September 2024*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 149–159. [CrossRef]
- Meier, J. Ensuring Inter-Model Consistency. Ph.D. Thesis, Carl von Ossietzky Universität Oldenburg, Oldenburg, Germany, 2024.
- Niepostyn, S.; Daszczuk, W.B. Automatization of Building IT Projects Using Composite Consistency Rules. *arXiv* **2026**, arXiv:2603.24726. [CrossRef]
- Erata, F.; Goknil, A.; Kurtev, I.; Tekinerdogan, B. Alloy in Ecore: Embedding of First-Order Relational Logic into Meta-Object Facility for Automated Model Reasoning. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2018), Lake Buena Vista, FL, USA, 4–9 December 2018*; Association for Computing Machinery: New York, NY, USA, 2018; pp. 920–923. [CrossRef]
- Wu, H.; Farrell, M. A Formal Approach to Finding Inconsistencies in a Metamodel. *Softw. Syst. Model.* **2021**, *20*, 1271–1298. [CrossRef]
- Zave, P.; Nelson, T. Validation of Formal Models: A Case Study. In *The Practice of Formal Methods*; Cavalcanti, A., Baxter, J., Eds.; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2024; Volume 14781. [CrossRef]
- Bordbar, B.; Anastasakis, K. UML2Alloy: A Tool for Lightweight Modelling of Discrete Event Systems. In *Proceedings of the IADIS International Conference on Applied Computing 2005 (AC 2005), Algarve, Portugal, 22–25 February 2005*; IADIS Press: Lisbon, Portugal, 2005; pp. 209–216.
- Sommerville, I. *Software Engineering*; Pearson: London, UK, 2016.

26. Siqueira, F.L.; Silva, P.S.M. An Essential Textual Use Case Meta-Model Based on an Analysis of Existing Proposals. In Proceedings of the Workshop em Engenharia de Requisitos (WER'11), Rio de Janeiro, Brazil, 28–29 April 2011.
27. UML Diagrams. Online Shopping UML Example. Available online: <https://www.uml-diagrams.org/examples/online-shopping-example.html> (accessed on 8 February 2026).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.